Stler Words

Technical Documentation

Version: 1.0 (April 2026)

Live URL: stler-words.vercel.app

Repository: github.com/JuliaSivridi/Words

Stack: React 18 · JavaScript · Vite 6 · Google Sheets API v4

> A personal vocabulary learning app delivered as a serverless PWA. Stler Words are studied through three progressively harder modes — **FlipCard → MultipleChoice → MatchingGrid** — with spaced repetition counters stored directly in Google Sheets. No backend, no subscription.

Table of Contents

1. Overview
2. Tech Stack
3. Project Architecture
4. Data Model
5. Session Algorithm
6. Screens & Navigation
7. Authentication
8. Google Sheets API Integration
9. UI & Theme
10. PWA & Mobile
11. Setup & Deployment

1. Overview

Stler Words is a personal vocabulary learning application delivered as a **Progressive Web App (PWA)**. It runs in any modern browser and installs on Android and iOS home screens as a standalone app. There is no dedicated backend server — all persistent data lives in a **Google Spreadsheet** (named `db_words`) found or created automatically via the Google Drive API v3.

Words are studied through three progressively harder exercise modes. Each mode has its own repetition counter per word. Once a word completes all three modes (24 total repetitions), it is automatically marked as learned and removed from future sessions.

Key Design Goals

- **Zero backend** — Google Sheets as a database; no server-side code
- **Any language pair** — each sheet tab is a separate language pair (e.g. `RU-EN`)
- **Spaced repetition** — three modes, each requiring progressively more repetitions
- **Category filter** — words tagged by category; sessions and word list filter by category
- **Cross-device sync** — selected language and category filter stored in the sheet
- **Mobile-first** — 16 px minimum font, touch-friendly, installable PWA
- **Dark mode** — follows OS preference; consistent color palette across all screens

2. Tech Stack

Layer	Technology	Version	Notes
UI framework	React	18	Hooks only; no class components
Language	JavaScript	ES2022	No TypeScript
Build tool	Vite	6	ES modules, HMR
Styling	CSS Modules + CSS variables	—	Light/dark via `prefers-color-scheme`
Routing	React Router DOM	6	Client-side; hash-free URLs
Database	Google Sheets API v4	—	Row-per-word; direct browser fetch
File discovery	Google Drive API v3	—	Find or create `db_words` spreadsheet
Auth	Google Identity Services	—	OAuth 2.0 implicit grant; silent sign-in
Offline storage	None	—	localStorage for language/category prefs only
PWA	Manual manifest + SW	—	manifest.json + sw.js; no Workbox
Hosting	Any static host	—	Vercel, Netlify, GitHub Pages

3. Project Architecture

The app is a single-page React application. All state is managed locally in React hooks and passed top-down from `App.jsx`. There are no global state stores — the component tree is shallow enough that prop drilling is manageable.

```
src/
├── App.jsx # root router, auth state, sheet ID, language & category
├── auth.js # GIS token client, trySilentSignIn(), onAuthChange()
├── sheetsApi.js # Drive + Sheets API fetch wrappers (all API calls)
├── constants.js # M1_MAX=4 M2_MAX=8 M3_MAX=12 TOTAL_REPS=24
├── langMap.js # tab name → display label mapping
├── theme.css # CSS variables, light/dark theme, global utilities
├── screens/
│ ├── LoginScreen.jsx # Google sign-in page
│ ├── HomeScreen.jsx # Start / Language / Category / Word List buttons
│ ├── SessionScreen.jsx # 12-step session orchestration + progress bar
│ ├── WordListScreen.jsx # scrollable word list with learned toggle
│ ├── LanguageScreen.jsx # tab selection from db_words spreadsheet
│ └── CategoryScreen.jsx # multi-select category filter
├── components/
│ ├── FlipCard.jsx # Mode 1: flip card, swipe support, Learn and hide
│ ├── MultipleChoice.jsx # Mode 2: 2×2 choice grid
│ ├── MatchingGrid.jsx # Mode 3: 6-pair matching columns
│ ├── NextButton.jsx # shared "Next →" button, sticky at bottom
│ ├── CheckIcon.jsx # shared Lucide-style checkmark SVG
│ └── WordListItem.jsx # single row in word list
└── hooks/
├── useWords.js # load words, saveSessionUpdates, setLearned, resetWord
└── useSession.js # buildSession() — 12-step session builder
```

Data Flow

1. **App start** → `trySilentSignIn()` → if ok: `findOrCreateWordsFile()` → `readSettings()`
2. Settings (language + category) restored from sheet, overriding localStorage values
3. `useWords(sheetId, tab)` fetches all rows for the selected tab via Sheets API
4. `buildSession(words, categoryFilter)` plans 12 steps; mode components render each step
5. User completes a step → counter incremented in memory (`pendingUpdates` ref)
6. After step 12 → `batchUpdateWords()` sends all counter updates to Sheets in one call
7. "Learn and hide" → immediate single-row update via `markLearned()`

4. Data Model

All data lives in the user's Google Spreadsheet named `db_words`. The app finds it in Google Drive by name and creates it automatically on first login. Each language pair is a separate sheet tab.

Word Tab (e.g. `RU-EN`)

Tab name format: `TRANSLATION-STUDY`. For `RU-EN` the user studies English; column C holds the Russian translation. Row 1 is treated as a header only if `B1 = "word"` (case-insensitive); otherwise all rows are data.

Col	Field	Type	Notes
A	category	string	Optional grouping label (e.g. "Verbs", "Food")
B	word	string	The word being studied (target language)
C	translation	string	Native language meaning
D	m1	number	FlipCard repetition count — graduates at 4
E	m2	number	MultipleChoice repetition count — graduates at 8
F	m3	number	MatchingGrid repetition count — auto-learned at 12
G	learned	`TRUE | FALSE`	Hides word from future sessions

`_settings` Tab

Stores cross-device preferences. Created automatically on first settings write.

Cell	Content
`A1`	Last selected language tab code (e.g. `RU-EN`)
`A2`	Category filter — comma-separated names, or empty string for "all"

> **Sync strategy:** Language and category are stored in both localStorage (for fast initial load) and the `_settings` tab (for cross-device sync). On login, sheet values win and overwrite localStorage.

Repetition Constants (`constants.js`)

Constant	Value	Meaning
`M1_MAX`	4	Reps needed to graduate from FlipCard mode

`M2_MAX`	8	Reps needed to graduate from MultipleChoice mode
`M3_MAX`	12	Reps needed to auto-mark word as learned
`TOTAL_REPS`	24	Total repetitions per word (4 + 8 + 12)

5. Session Algorithm

Each session consists of exactly 12 steps, built by `buildSession(words, categoryFilter)` in `src/hooks/useSession.js`. Counters are accumulated in memory and batch-saved after the final step.

Pool Building

Three word pools are built from active (non-learned) words that match the category filter:

Pool	Eligibility condition
mode1Pool	`m1 < 4`
mode2Pool	`m1 ≥ 4` AND `m2 < 8`
mode3Pool	`m2 ≥ 8` AND `m3 < 12` AND at least 6 words

Step Planning

Available modes receive equal shares of the 12 steps. Unavailable modes donate their slots evenly to the others. The resulting plan is shuffled so modes are interleaved:

Available modes	Step distribution
All three (1, 2, 3)	4 + 4 + 4
Two modes (any)	6 + 6
One mode only	12

Word Selection per Mode

- **Mode 1 & 2** — Shuffled queue: all words in the pool are cycled before any word repeats. Prevents the same word appearing multiple times in one session.
- **Mode 3** — Pool sorted by `m3` ascending, then sliced into non-overlapping groups of 6. Each mode 3 step uses the next group (`m3i % numGroups`), so different words appear on each occurrence.

Cross-Category Supplements (`buildMode3Pool`)

If a selected category has no words that qualify for mode 3 yet (i.e. no words with `m2 ≥ 8`), the app supplements the pool with the most-advanced unlearned words from that category. This ensures every selected category is represented in the matching grid even when the user is still early in their learning progression.

6. Screens & Navigation

All routing is handled by React Router DOM v6 (`BrowserRouter`). Routes: `/`, `/session`, `/words`, `/language`, `/category`.

LoginScreen

- Horizontal logo row: app icon (icon-192.png) + "Words" heading + tagline
- Two description lines explaining Google Sheets storage and offline capability
- Orange "Sign in with Google" button — triggers GIS popup flow

HomeScreen

- Top bar: app name + user avatar (tapping signs out)
- Four buttons in order: **Start** (primary CTA) → Language → Category → Word List
- Language and Category buttons show a sub-label (selected value or "All categories")
- All buttons use monochrome Lucide-style SVG icons (`currentColor`)

SessionScreen

- Top bar: back (✕) button + step counter ("3 / 12") + thin progress bar
- Renders FlipCard, MultipleChoice, or MatchingGrid depending on `step.mode`
- NextButton always visible at the bottom (sticky via `margin-top: auto` in flex column)
- After step 12: calls `batchUpdateWords()`, shows "Session complete!" with checkmark

FlipCard (Mode 1)

- CSS 3D flip on X axis (`rotateX(180deg)`) with 0.5 s cubic-bezier transition
- Front: word + "tap to flip" hint. Back: translation on surface-colored background
- Triggers: click / tap / vertical swipe (delta Y > 50 px via `touchstart` / `touchend`)
- "Learn and hide" button: marks word as learned immediately, unlocks Next →
- Next → unlocks after first flip OR after "Learn and hide"

MultipleChoice (Mode 2)

- Top half: "Learn and hide" button + large word card
- Bottom half: 2×2 grid of translation choice cards
- Wrong tap: red flash animation (`flash-wrong` keyframe), question stays active
- Correct tap: card turns green, Next → unlocks
- Wrong answers drawn from the same category-filtered pool; falls back to full word list if < 3 options

MatchingGrid (Mode 3)

- 6 word–translation pairs; left column = shuffled words, right column = shuffled translations
- Bidirectional tap: either column can be tapped first to select; tapping opposite column attempts match
- Correct pair: both cards turn green + permanently deactivate
- Wrong pair: both flash red, selection resets
- All 6 pairs matched → Next → unlocks. No "Learn and hide" in this mode

LanguageScreen

- Lists all sheet tabs from `db_words` (excludes `Sheet1`, `Лист1`, `_settings`)
- Selected tab shown with orange checkmark on right (shared `<CheckIcon>` component)
- Tapping a tab saves to localStorage + `_settings` and navigates back to Home

CategoryScreen

- Multi-select: any combination of categories can be active simultaneously
- "All categories" row at top — toggles between full selection and empty selection
- Back button saves selection and navigates to Home (no separate "Done" button)
- Stored as `null` (all) or `string[]`; synced to sheet as comma-separated string

WordListScreen

- Header shows "learned / total" counter for visible words
- Each row: word | translation | progress ("3 / 24") | toggle icon
- Eye icon: tap to mark as learned (saved immediately). Eye-crossed: tap to reset all counters
- Filtered by `sessionCategory` if set

7. Authentication

Authentication uses **Google Identity Services (GIS)** with the OAuth 2.0 implicit token grant. No redirect URIs are needed — only Authorized JavaScript Origins must be configured.

- On app load: `trySilentSignIn()` calls `tokenClient.requestAccessToken({ prompt: '' })`
- Empty `prompt` = no UI if user previously granted access; popup only on first login
- User profile (`name`, `email`, `picture`) fetched from Google userinfo endpoint after token received
- Email saved in localStorage as `login_hint` for subsequent silent sign-ins
- Access token stored in module-level memory variable only (never persisted)
- On 401 from Sheets API: silent token refresh attempted automatically
- Sign-out: clears user object and calls `google.accounts.oauth2.revoke()`

Required OAuth Scopes

- `email`, `profile`
- `https://www.googleapis.com/auth/spreadsheets`
- `https://www.googleapis.com/auth/drive.metadata.readonly`

> **Google Cloud Console setup:** Enable Google Sheets API v4 *and* Google Drive API. Create an OAuth 2.0 Client ID (Web application type). Add your domain to *Authorized JavaScript Origins* (not Redirect URIs). Add your Google account as a test user in the OAuth consent screen.

8. Google Sheets API Integration

All API calls are plain `fetch()` requests in `src/sheetsApi.js`, using the in-memory access token. A central `request(url, options)` helper attaches the Bearer header and calls `refreshTokenIfNeeded()` before every request.

Key API Calls

Operation	Endpoint
Find spreadsheet	`GET drive/v3/files?q=name='db_words'&mimeType=...spreadsheet...`
Create spreadsheet	`POST sheets/v4/spreadsheets` with `{ title: 'db_words' }`
List tabs	`GET sheets/v4/spreadsheets/{id}?fields=sheets.properties.title`
Read words	`GET sheets/v4/spreadsheets/{id}/values/{tab}!A1:G`
Read settings	`GET sheets/v4/spreadsheets/{id}/values/_settings!A1:A2`
Batch update counters	`POST sheets/v4/spreadsheets/{id}/values:batchUpdate` — range `{tab}!D{row}:G{row}`
Mark learned	`PUT sheets/v4/spreadsheets/{id}/values/{tab}!G{row}`
Reset word	`PUT sheets/v4/spreadsheets/{id}/values/{tab}!D{row}:G{row}` — `[[0, 0, 0, "FALSE"]]`
Write settings	`PUT sheets/v4/spreadsheets/{id}/values/_settings!A1:A2`

Batch Save Strategy

- Counter increments are accumulated in a `pendingUpdates` ref during the session
- `batchUpdateWords()` is called once after step 12 — one API call for all 12 updates
- "Learn and hide" triggers an immediate single-row update (`markLearned()`)
- `resetWord()` writes `[[0, 0, 0, "FALSE"]]` to columns D–G to clear all counters
- Sheet ID cached in localStorage under `words_sheet_id` to avoid Drive API calls on repeat visits

9. UI & Theme

Color System

All colors are CSS variables defined in `src/theme.css`. Dark mode switches automatically via `@media (prefers-color-scheme: dark)`.

Variable	Light	Dark	Usage
`--bg`	#FFFFFF	#1C1C1C	App background
`--surface`	#F3F1EF	#363636	Headers, secondary backgrounds
`--surface-2`	#E8E8E8	#424242	Hover states
`--text`	#1C1C1C	#EFEFEF	Primary text
`--text-muted`	#6B6B6B	#949494	Secondary text, counters
`--accent`	#E07E38	#E8935A	Buttons, checkmarks, progress bar
`--accent-hover`	#C96E2F	#D4804A	Hover on accent elements
`--success`	#6BBF7A	#5AA569	Correct answers, learned state
`--error`	#E07070	#C96060	Wrong answers
`--border`	#E0E0E0	#4A4A4A	Card borders, dividers
`--card-bg`	#FFFFFF	#363636	Flashcard backgrounds

> **Consistency:** The accent color (#E07E38) is intentionally identical to the primary color in the companion Tasks project (`HSL 25 75% 55%`). In dark mode the accent is lightened to #E8935A for better contrast on dark backgrounds.

Icons

All icons are inline SVGs in the Lucide style: `viewBox="0 0 24 24"`, `stroke="currentColor"`, `strokeWidth="1.75"`, `strokeLinecap="round"`, `strokeLinejoin="round"`. A shared `<CheckIcon size={n}>` component is used for all checkmark indicators (language selection, category selection, word list learned state, session complete screen). No icon library package is installed — paths are inlined directly.

Touch & Mobile

- Minimum tap target: 44 px on all interactive elements

- `-webkit-tap-highlight-color: transparent` on all buttons
- FlipCard: vertical swipe (delta Y > 50 px) triggers flip
- All screens: `height: 100dvh; overflow: hidden` — no page-level scroll
- Content that may exceed viewport (word list) uses internal `overflow-y: auto`

10. PWA & Mobile

- Web App Manifest (`public/manifest.json`): name "Words", display `standalone`
- Theme color: `#E07E38` in both `manifest.json` and `<meta name="theme-color">`
- Icons: `icon-192.png` and `icon-512.png` — orange rounded square with white "W"
- Service worker (`public/sw.js`): App Shell caching for HTML, JS, CSS, icons
- GIS script (`accounts.google.com/gsi/client`) loaded with `async defer` in `index.html`
- Android: Chrome auto-prompts for installation or browser menu → Install app
- iOS: Safari → Share button → Add to Home Screen

11. Setup & Deployment

Prerequisites

- Google account
- Google Cloud project with **Google Sheets API v4** and **Google Drive API** enabled
- OAuth 2.0 Client ID (type: Web application)
- Node.js ≥ 18

Google Cloud Console

1. Enable **Google Sheets API v4** and **Google Drive API**
2. Create an **OAuth 2.0 Client ID** → Web application
3. Add to **Authorized JavaScript Origins** (not Redirect URIs):

   ```
   http://localhost:5173
   https://your-app.vercel.app
   ```

4. Add your account as a **test user** in the OAuth consent screen

Prepare Your Google Sheet

The app automatically creates the `db_words` spreadsheet on first login. You only need to add a tab and populate it:

1. Open the created `db_words` spreadsheet in Google Drive
2. Add a tab named like `RU-EN` (format: `TRANSLATION-STUDY`)
3. Fill columns: **A** = category (optional), **B** = word, **C** = translation
4. Row 1 can be a header or data — the app auto-detects based on whether B1 = "word"

Local Development

```
git clone https://github.com/JuliaSivridi/Words.git
cd Words
npm install
```

Create `.env` in the project root:

```
VITE_GOOGLE_CLIENT_ID=your-client-id.apps.googleusercontent.com
```

```
npm run dev    # http://localhost:5173
npm run build  # production build → dist/
```

Deploy to Vercel

1. Import the repository at **vercel.com**
2. Add environment variable in Settings → Environment Variables: `VITE_GOOGLE_CLIENT_ID`
3. Every push to `main` triggers automatic deployment
4. Add the Vercel URL to Google Cloud Console Authorized JavaScript Origins

> **Single environment variable:** Unlike the companion Tasks project, Words requires only `VITE_GOOGLE_CLIENT_ID`. The spreadsheet ID is discovered automatically via the Drive API and cached in localStorage — no `VITE_SPREADSHEET_ID` needed.